

MEMORIA

EJERCICIO EVALUABLE 2



Sistemas distribuidos

Grupo 80- Curso 25/26

Nombres, apellidos y NIA
Alumno 1: Daniel Santos Méndez (100522786) Grupo 80 - Grado en Ingeniería Informática
Alumno 2: Sergio Aladro Municio (100522360) Grupo 80 - Grado en Ingeniería Informática

Parte 1. Arquitectura general del sistema

El sistema implementa un servicio de almacenamiento de tuplas clave-valor distribuido en tres capas que se comunican exclusivamente mediante sockets TCP. Cada capa tiene una responsabilidad clara e independiente del lenguaje de programación utilizado:

Capa	Fichero	Responsabilidad
Aplicación Cliente	app-cliente.c	Invoca la API definida en claves.h sin nada que conocer.
Proxy (Biblioteca Cliente)	proxy-sock.c y libproxyclaves.c	Serializa cada llamada a la API, abre una conexión TCP persistente con el servidor y deserializa la respuesta.
Servidor TCP concurrente	servidor-sock.c y libclaves.c	Acepta conexiones, crea un hilo por cliente, deserializa la petición, ejecuta la operación.

• **Decisión de diseño:** conexión TCP. El proxy mantiene un único socket abierto (sd global). El servidor atiende todas las peticiones de ese cliente en un bucle, sin cerrar la conexión entre peticiones. Esto reduce la latencia y el número de handshakes TCP respecto a abrir/cerrar por cada llamada.

1. Compilación y ejecución

El proyecto incluye un archivo **Makefile** que automatiza la generación de las bibliotecas dinámicas (**libclaves.so** y **libproxyclaves.so**) y los ejecutables. Para compilar el proyecto completo, basta con ejecutar el comando **make** en la raíz del directorio. Para iniciar el servicio, el servidor debe ejecutarse indicando el puerto de escucha por línea de comandos: **\$> ./servidor <PUERTO>** (ej. **./servidor 4500**). En el lado del cliente, es necesario definir la dirección IP y el puerto del servidor mediante variables de entorno antes de ejecutar las pruebas: **\$> export IP_TUPLAS=localhost \$> export PORT_TUPLAS=4500**. Una vez exportadas, se pueden ejecutar los clientes de prueba (**./cliente** o **./cliente2**).

Parte 2. Protocolo de aplicación (proxy/servidor)

1. Cabecera de petición (5 bytes)

Bytes	Tipo	Descripción
0	uint8	Código de operación (1 - 6)
1 - 4	int32	Longitud en bytes del cuerpo que sigue

2. Cabecera de respuesta (9 bytes)

Bytes	Tipo	Descripción
0	uint8	Código de operación (eco del recibido)
1 - 4	int32	Estado 0: éxito,-1: error
5 - 8	uint32	Longitud del cuerpo de respuesta (si hay)

Parte 3. Implementación proxy-sock.c

1. Función de conexión lazy

Todas las funciones comprueban if (sd == -1) antes de actuar. Si el socket no está abierto, llaman a la función interna conectar() que lee **IP_TUPLAS** y **PORT_TUPLAS** mediante getenv(), resuelve el nombre con gethostbyname() (válido tanto para direcciones decimal-punto como dominio-punto), crea el socket y llama a connect(). Si falla en cualquier punto, devuelve -1 y cierra el descriptor.

2. Serialización independiente del lenguaje

Todos los enteros y floats se convierten a big-endian antes de enviarlos mediante funciones auxiliares internas (**escribir_entero_32**, **escribir_float_32**, **leer_entero_32**, **leer_float_32**). Los floats se serializan copiando sus 4 bytes IEEE 754 a un entero de 32 bits y aplicando **htonl()**. Esto garantiza que el protocolo funcione entre arquitecturas distintas y podría implementarse en cualquier otro lenguaje.

3. Envío y recepción fiables

Las funciones **enviar_todo()** y **recibir_todo()** garantizan que se transmiten exactamente N bytes usando un bucle, ya que **send()** y **recv()** de TCP pueden devolver menos bytes de los solicitados. Si en cualquier llamada se detecta un error o cierre de conexión, el socket se cierra y {C('sd')} se pone a -1 para forzar reconexión en la siguiente llamada.

4. Particularidad del get_value

Es la única operación que recibe un cuerpo de respuesta no vacío. Tras leer la cabecera y comprobar que el estado es 0, el proxy lee exactamente longitud_cuerpo bytes adicionales y los deserializa en el orden: longitud de value1, bytes de value1, N_value2, N floats, x, y, z de value3.

Parte 4. Implementación servidor-sock.c

El servidor implementa la parte que recibe y atiende las peticiones enviadas por el proxy a través de sockets TCP. Su función principal es escuchar en un puerto, aceptar conexiones de clientes, leer e interpretar cada petición, ejecutar la operación correspondiente sobre el servicio de claves y devolver una respuesta. Además, el servidor está diseñado para funcionar de forma concurrente, atendiendo cada conexión en un hilo independiente.

1. Inicialización del servidor y socket de escucha

El servidor recibe el puerto por línea de comandos y crea un socket TCP con **socket()**. Después configura la opción **SO_REUSEADDR** con **setsockopt()**, asocia el socket al puerto con **bind()** y lo deja en espera de conexiones con **listen()**. También instala manejadores de señal para poder

cerrarse de forma controlada si recibe **SIGINT** o **SIGTERM**, cerrando el socket principal y saliendo correctamente.

2. Concurrencia mediante hilos bajo demanda

Cada vez que llega una nueva conexión mediante **accept()**, el servidor crea un hilo nuevo con **pthread_create()** para atender a ese cliente. El hilo se marca como detached, de forma que no sea necesario hacer **pthread_join()** después. Este mecanismo corresponde a un modelo de hilos bajo demanda, ya que los hilos no existen previamente, sino que se crean solo cuando llega una petición. Así, el servidor puede atender varios clientes al mismo tiempo.

3. Lectura y validación de la petición

Cada hilo ejecuta la función que atiende al cliente. Primero lee la cabecera de la petición, que ocupa 5 bytes: 1 byte para el código de operación y 4 bytes para la longitud del cuerpo. Si la longitud es negativa o supera **MAX_CUERPO**, la petición se considera inválida y el servidor responde con error. Si existe cuerpo, se reserva memoria dinámica y se lee completo con **recibir_todo()**, asegurando que se reciban exactamente todos los bytes esperados.

4. Interpretación del mensaje y ejecución de la operación

Una vez leída la petición, el servidor interpreta el contenido según el código de operación. Para destroy no hay cuerpo; para **delete_key**, **exist** y **get_value** solo se recibe la clave; y para **set_value** y **modify_value** se reciben la clave, **value1**, **N_value2**, el vector de floats y la estructura Paquete. Tras validar el formato, el servidor llama a la función correspondiente de claves.c mediante la función interna **ejecutar_peticion()**. De esta forma, la lógica de red queda separada de la lógica del servicio.

5. Construcción y envío de la respuesta

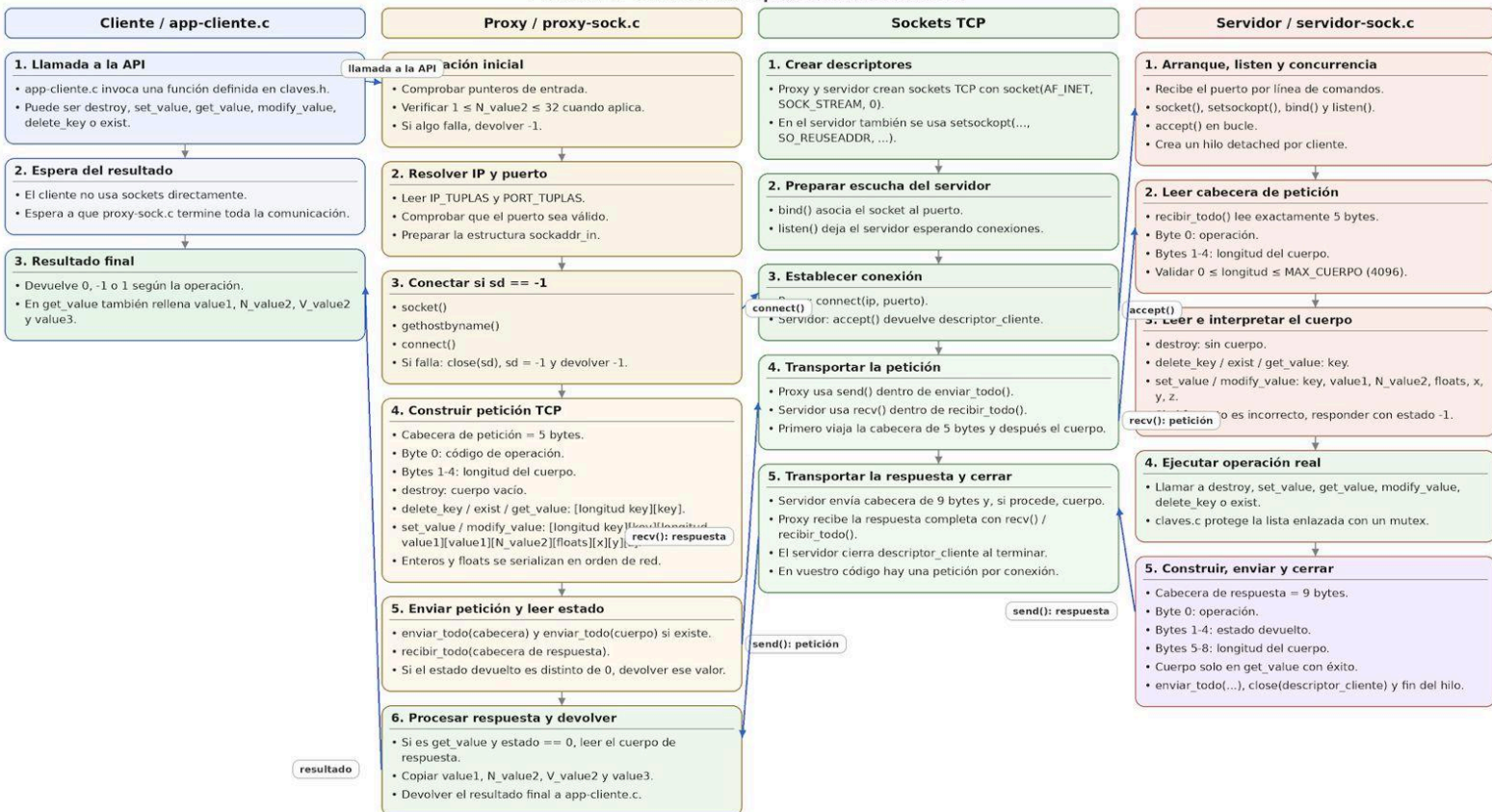
Después de ejecutar la operación, el servidor construye la respuesta TCP. La cabecera de respuesta ocupa 9 bytes: 1 byte para la operación, 4 bytes para el estado devuelto y 4 bytes para la longitud del cuerpo de respuesta. Normalmente la respuesta no lleva cuerpo, salvo en **get_value** cuando la operación termina correctamente. En ese caso, el servidor construye un cuerpo adicional con **value1**, **N_value2**, los floats y los campos x, y y z. Finalmente, envía la cabecera y el cuerpo con **enviar_todo()**, libera la memoria utilizada, cierra la conexión del cliente y termina el hilo.

Parte 5. Diagrama de flujo

El diagrama representa el funcionamiento completo del sistema, desde que **app-cliente.c** llama a una función de la API hasta que el servidor procesa la petición y devuelve la respuesta. Para que se entienda mejor, se divide en cuatro partes: cliente, proxy, sockets TCP y servidor. El cliente no usa sockets directamente, sino que llama a las funciones de **claves.h**, y es **proxy-sock.c** quien valida los datos, obtiene la IP y el puerto, construye la petición TCP y la envía. Esta petición tiene una cabecera fija de 5 bytes y un cuerpo variable según la operación. Después, el proxy recibe la respuesta y devuelve el resultado al cliente. La parte de sockets muestra la comunicación TCP entre ambos lados. En el cliente se usan **socket()** y **connect()**, y en el servidor **socket()**, **bind()**, **listen()** y **accept()**. El envío y la recepción se hacen con **send()** y **recv()**, apoyándose en funciones auxiliares para asegurar que se transmitan todos los bytes necesarios. En el servidor, primero se lee la cabecera, luego el cuerpo y después se interpreta la operación. A continuación, se llama a la función correspondiente de claves.c y se construye la respuesta TCP, que tiene una cabecera fija de 9 bytes y, en **get_value**, puede incluir también un cuerpo con los datos obtenidos. La concurrencia se ha resuelto con hilos bajo demanda, es decir, creando un hilo nuevo por cada conexión recibida. Además, en claves.c se utiliza un mutex global para proteger la estructura compartida y evitar condiciones de carrera.

Diagrama de flujo del protocolo cliente / proxy / sockets / servidor

Práctica 2 · Servicio de tuplas sobre sockets TCP



Parte 6. Pruebas y validación

- **app-cliente.c (Pruebas Secuenciales):** Este programa realiza una batería de 15 pruebas exhaustivas. Comienza limpiando la base de datos con **destroy()**. A continuación, realiza inserciones (**set_value**), lecturas (**get_value**), modificaciones (**modify_value**) y borrados (**delete_key**), comprobando en cada paso que la operación ha tenido éxito. También incluye pruebas de manejo de errores, como intentar insertar un valor con tamaño excesivo o modificar una clave inexistente, verificando que el servidor rechaza la petición sin colapsar.
- **app-cliente-2.c (Prueba de Concurrencia):** Este programa está diseñado para someter al servidor a estrés y validar que no existen condiciones de carrera. Utilizando la llamada al sistema **fork()**, genera 5 procesos cliente que envían peticiones simultáneas de escritura y lectura al servidor. Los resultados de esta prueba demuestran que el servidor atiende correctamente peticiones entrelazadas, y que el cerrojo (mutex) implementado en la lista enlazada de claves.c protege la integridad de los datos, permitiendo que cada proceso recupere sus valores correctos sin interferencias.

Parte 7. Conclusión del ejercicio

En general, esta práctica nos ha servido para entender mucho mejor cómo funciona un sistema distribuido de verdad usando sockets TCP. A partir de lo que ya habíamos hecho en la práctica anterior, en esta hemos podido centrarnos más en la comunicación entre cliente y servidor, en cómo se diseña un protocolo y en cómo se gestiona la concurrencia. En conjunto, creemos que ha sido una práctica útil porque nos ha ayudado a ver de forma más clara cómo se conectan todas las partes del sistema y cómo hacer que funcionen de forma correcta y ordenada.